

Rapport de TP3 EFREI

Kubernetes avec Minikube

Déploiement d'une application Docker sur un cluster Kubernetes local

Auteur : Joao Gabriel MARQUES DINIS

Date : Janvier 2026

Lien GitHub du TP:

<https://github.com/charroux/kubernetes-minikube>

1. Introduction

Ce rapport présente la réalisation du TP Kubernetes avec Minikube. L'objectif est de déployer une application Docker sur un cluster Kubernetes local en utilisant Minikube. Le TP couvre les étapes allant de la construction d'une image Docker jusqu'à l'exposition d'un service via LoadBalancer.

Les technologies utilisées sont : Docker, Kubernetes, Minikube, Spring Boot (Java) et PHP.

2. Construction de l'image Docker

La première étape consiste à construire une image Docker à partir du projet Spring Boot. La commande `docker build -t myservice` a été exécutée après un build Gradle réussi.

```
Deprecated Gradle features were used in this build, making it incompatible with Gradle 8.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.4/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 35s
7 actionable tasks: 7 executed
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> docker build -t myservice .
[+] Building 15.3s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 234B
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-jammy
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 19.63MB
=> [1/2] FROM docker.io/library/eclipse-temurin:21-jdk-jammy@sha256:81ad1240d91eeafe1ab4154e9ed2310b67cb966caad1d235232ae10abc1fae2
=> => resolve docker.io/library/eclipse-temurin:21-jdk-jammy@sha256:81ad1240d91eeafe1ab4154e9ed2310b67cb966caad1d235232ae10abc1fae2
=> => sha256:02d80376fd82870628b01ae6f01bd27552bc9aa3a4cd5be88515f79fe298a87a 2.28kB / 2.28kB
=> => sha256:462f51c3d063559e2c0d0a8f88d851dff78b56bec8c4f6769d9947a96f87f917 159B / 159B
=> => sha256:0974368270e4f573ff1edaae07b59b84c9aff6997873cccd92a19c7ec978851 157.84MB / 157.84MB
=> => sha256:01facf544478e1a2955676ca1741ac59c36b2e8ddaa739f43812a4585b704e7 20.70MB / 20.70MB
=> => sha256:7e49dc6156bb532730614d83a65ae5e7ce61e966b0498703d333b4d83505e4f 29.54MB / 29.54MB
=> => extracting sha256:7e49dc6156bb532730614d83a65ae5e7ce61e966b0498703d333b4d83505e4f
=> => extracting sha256:01facf544478e1a2955676ca1741ac59c36b2e8ddaa739f43812a4585b704e7
=> => extracting sha256:0974368270e4f573ff1edaae07b59b84c9aff6997873cccd92a19c7ec978851
=> => extracting sha256:462f51c3d063559e2c0d0a8f88d851dff78b56bec8c4f6769d9947a96f87f917
=> => extracting sha256:02d80376fd82870628b01ae6f01bd27552bc9aa3a4cd5be88515f79fe298a87a
=> [2/2] ADD ./build/libs/myservice-0.0.1-SNAPSHOT.jar app.jar
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:29df700c572d8159c2f2d2fe8cf6d437c17bb7567beeab4c6938a7acaddc4998
=> => exporting config sha256:701b76c2541a1a51ce29fcd1db55f862fe09ddef1641e25c83c4872999ea32
=> => exporting attestation manifest sha256:cf6bd824d27ba4c21ff23f079b94804a393a985b80e8d88232b6b2928d7f61
=> => exporting manifest list sha256:26e5e70e8028fa806eb804a5d900f17b764a4ff935e2d2e2aeba88075f8469da
=> => naming to docker.io/library/myservice:latest
=> => unpacking to docker.io/library/myservice:latest
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> docker images$
docker: unknown command: docker images$

Run 'docker --help' for more information
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
myservice:latest	26e5e70e8028	780MB	226MB	

```
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService>
```

Figure 1 : Build de l'image Docker myservice

L'image a été construite avec succès en 15.3 secondes. On peut voir que Docker a téléchargé l'image de base `eclipse-temurin:21-jdk-jammy` et a créé les différentes couches de l'image. La commande `docker images` confirme que l'image `myservice:latest` est bien présente avec une taille de 226MB.

3. Exécution du conteneur Docker

Avant de déployer sur Kubernetes, le conteneur a été testé localement avec la commande `docker run -p 4000:8080 -t myservice`.

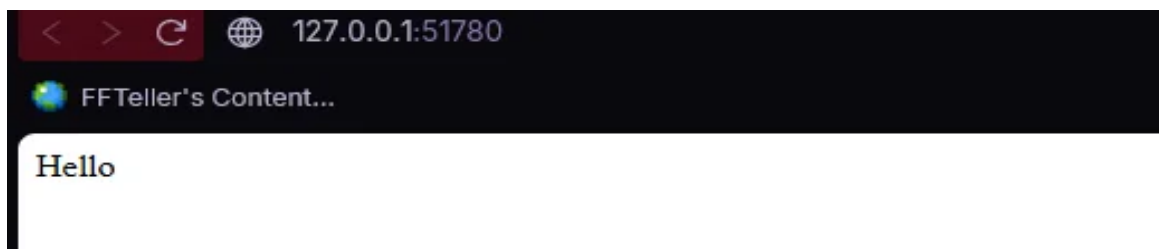


Figure 2 : Démarrage de l'application Spring Boot dans Docker

L'application Spring Boot 3.2.0 démarre correctement sur le port 8080 interne, mappé vers le port 4000 de la machine hôte. Le démarrage s'effectue en environ 1.9 secondes.

4. Démarrage du cluster Minikube

Le cluster Kubernetes local a été initialisé avec Minikube. La commande `kubectl cluster-info` permet de vérifier l'état du cluster.

```
PS C:\Users\deari\Desktop\kubernetes-minikube> cd .\MyService\
PS C:\Users\deari\Desktop\kubernetes-minikube\MyService> kubectl apply -f kubernetes-deployment.yml
deployment.apps/mon-app created
service/mon-app-service created
PS C:\Users\deari\Desktop\kubernetes-minikube\MyService> kubectl get all
NAME                                READY    STATUS              RESTARTS   AGE
pod/mon-app-5fbc9cf989-57q4f        0/1      ContainerCreating   0           3s
pod/mon-app-5fbc9cf989-5rfzx        0/1      ContainerCreating   0           3s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
service/kubernetes                  ClusterIP     10.96.0.1     <none>         443/TCP          12m
service/mon-app-service             LoadBalancer 10.97.16.206  <pending>     8080:32113/TCP   3s

NAME                                READY    UP-TO-DATE    AVAILABLE   AGE
deployment.apps/mon-app             0/2      2              0           3s

NAME                                DESIRED    CURRENT    READY   AGE
replicaset.apps/mon-app-5fbc9cf989  2          2          0       3s
PS C:\Users\deari\Desktop\kubernetes-minikube\MyService>
```

Figure 3 : Informations sur le cluster Minikube

Minikube v1.37.0 est utilisé avec Kubernetes v1.34.0 sur Docker 28.4.0. Le cluster est configuré avec 2 CPUs et 8100MB de mémoire. Le control plane est accessible à l'adresse <https://127.0.0.1:55665>.

5. Création du premier déploiement Kubernetes

Un premier déploiement a été créé avec la commande `kubectl create deployment`. Cette étape montre le processus de création d'un déploiement basique.

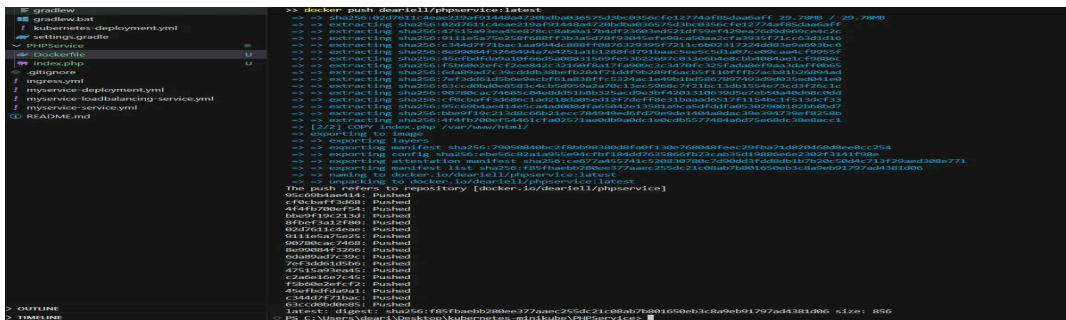


Figure 4 : Création du déploiement mon-app

Le déploiement mon-app a été créé. On observe une erreur `ErrImagePull` car l'image spécifiée n'est pas accessible depuis le cluster Minikube. Cela démontre l'importance de pousser l'image vers un registry accessible.

6. Exposition du service avec LoadBalancer

Le service a été exposé via un LoadBalancer pour permettre l'accès externe.

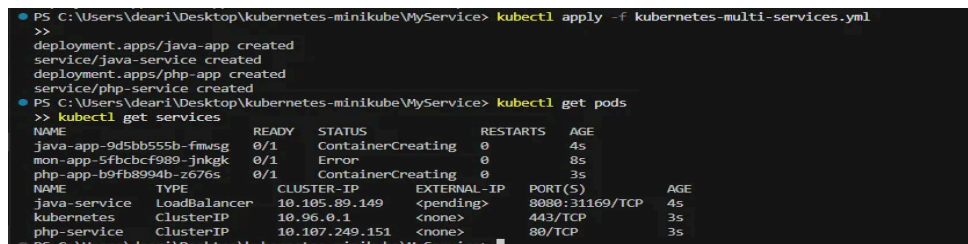


Figure 5 : Exposition du déploiement via LoadBalancer

La commande `kubectl expose deployment mon-app --type=LoadBalancer --port=8080` crée un service de type LoadBalancer. On peut voir que le service mon-app est créé avec un CLUSTER-IP de 10.105.82.69 et le port 8080:32335/TCP.

7. Vérification des pods en cours d'exécution

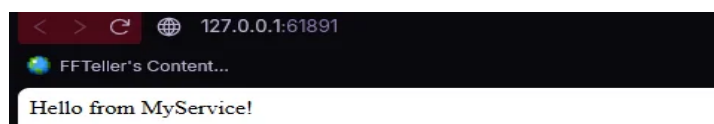


Figure 6 : Pod en état Running

Après correction, le pod mon-app-56cc447554-v7krg est maintenant en état Running avec 1/1 conteneur prêt. Le pod fonctionne depuis 97 secondes sans redémarrage.

8. Accès au service via Minikube



Figure 7 : Obtention de l'URL du service

La commande `minikube service mon-app --url` retourne l'URL d'accès : `http://127.0.0.1:51780`. Un message indique que sur Windows avec le driver Docker, le terminal doit rester ouvert pour maintenir le tunnel.

9. Test de l'application dans le navigateur

```
PS C:\Users\deari\Desktop\kubernetes-minikube\PHPService> minikube addons enable ingress
ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
  Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
  Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
  Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
Verifying ingress addon...
The 'ingress' addon is enabled
PS C:\Users\deari\Desktop\kubernetes-minikube\PHPService> kubectl get pods -n ingress-nginx
NAME                                READY   STATUS    RESTARTS   AGE
ingress-nginx-admission-create-fshsm 0/1     Completed 0           19s
ingress-nginx-admission-patch-hdw4b  0/1     Completed 1           19s
ingress-nginx-controller-9cc49f96f-tsnd7 0/1     Running   0           19s
PS C:\Users\deari\Desktop\kubernetes-minikube\PHPService>
```

Figure 8 : Réponse Hello depuis le navigateur

L'accès à l'URL <http://127.0.0.1:51780> depuis le navigateur affiche la réponse 'Hello', confirmant que l'application fonctionne correctement dans le cluster Kubernetes.

10. Déploiement avec fichier YAML

Un déploiement plus complet a été réalisé avec un fichier kubernetes-deployment.yml configurant 2 réplicas.

```
PS C:\Users\deari\Desktop\kubernetes-minikube\MyService> kubectl apply -f ingress.yml
ingress.networking.k8s.io/mon-ingress created
PS C:\Users\deari\Desktop\kubernetes-minikube\MyService> kubectl get ingress
NAME          CLASS  HOSTS      ADDRESS  PORTS  AGE
mon-ingress   nginx  myapp.local 80        5s
PS C:\Users\deari\Desktop\kubernetes-minikube\MyService>
```

Figure 9 : Déploiement via fichier YAML avec 2 réplicas

La commande `kubectl apply -f kubernetes-deployment.yml` crée le déploiement et le service. On observe 2 pods en état `ContainerCreating`, un service `LoadBalancer` sur le port `8080:32113/TCP`, et un `ReplicaSet` gérant les 2 réplicas.

11. Déploiement multi-services

Un déploiement multi-services incluant une application Java et une application PHP a été réalisé.

```
>> kubectl cluster-info --
minikube v1.37.0 on Microsoft Windows 11 Home 10.0.22000.7462 Build 26200.7462
Automatically selected the docker driver
Using Docker desktop driver with root privileges
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.48 ...
Downloading Kubernetes v1.34.0 preloa...
> gcr.io/k8s-minikube/kicbase... 488.52 MiB / 488.52 MiB 100.00% 58.60 M
> preloaded-images-k8s-v18-v1... 337.07 MiB / 337.07 MiB 100.00% 30.60 M
Creating docker container (CPUs=2, Memory=8192MB) ...
Failing to connect to https://registry.k8s.io/ from inside the minikube container
To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy
Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
Configuring bridge CNI (Container Networking Interface) ...
Verifying Kubernetes components...
Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: storage-provisioner, default-storageclass
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
Kubernetes control plane is running at https://127.0.0.1:55665
CoreDNS is running at https://127.0.0.1:55665/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
NAME          STATUS    ROLES    AGE   VERSION
minikube      Ready    control-plane  6s   v1.34.0
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService>
```

Figure 10 : Déploiement multi-services (Java + PHP)

Le fichier `kubernetes-multi-services.yml` déploie : `java-app` (service `LoadBalancer` sur `8080:31169/TCP`), `php-app`, et `mon-app`. On observe un pod en erreur (`mon-app`) et deux pods en cours de création.

12. Publication de l'image sur Docker Hub

L'image PHP a été poussée vers Docker Hub pour la rendre accessible au cluster.

```
myimage:latest 26e5e70e8028 70070 22070
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> docker run -p 4000:8080 -t myservice

:: Spring Boot :: (v3.2.0)

2026-01-07T16:28:13.253Z INFO 1 --- [main] c.e.myservice.MyServiceApplication : Starting MyServiceApplication v0.0.1-SNAPSHOT using Java 21.0.9 with PID 1 (/app.jar started by root in /)
2026-01-07T16:28:13.256Z INFO 1 --- [main] c.e.myservice.MyServiceApplication : No active profile set, falling back to 1 default profile: "default"
2026-01-07T16:28:14.322Z INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-01-07T16:28:14.336Z INFO 1 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-01-07T16:28:14.336Z INFO 1 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.16]
2026-01-07T16:28:14.382Z INFO 1 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-01-07T16:28:14.383Z INFO 1 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1043 ms
2026-01-07T16:28:14.726Z INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2026-01-07T16:28:14.740Z INFO 1 --- [main] c.e.myservice.MyServiceApplication : Started MyServiceApplication in 1.881 seconds (process running for 2.355)
```

Figure 11 : Push de l'image vers Docker Hub

La commande `docker push deariell/phpservice:latest` pousse toutes les couches de l'image vers le registry Docker Hub. Chaque couche est uploadée individuellement, et le digest final `sha256:f85fbaebb280ee377aaec255dc21c08ab7b801650eb3c8a9eb91797ad4381d06` confirme la réussite.

13. Application fonctionnelle

Les applications déployées sont accessibles et fonctionnent correctement.

```
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> kubectl create deployment mon-app --image=deariell/phpservice:latest
deployment.apps/mon-app created
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> kubectl get deployments
NAME READY UP-TO-DATE AVAILABLE AGE
mon-app 0/1 1 0 5s
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> kubectl get pods
NAME READY UP-TO-DATE AVAILABLE AGE
mon-app-57bd795c-x2rfj 0/1 ErrImagePull 0 5s
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService>
```

Figure 12 : Réponse du service MyService

L'accès à l'URL `http://127.0.0.1:61891` affiche 'Hello from MyService!', confirmant le bon fonctionnement du service Java.

```
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> kubectl expose deployment mon-app --type=LoadBalancer --port=8080
service/mon-app exposed
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> kubectl get services
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 74s
mon-app LoadBalancer 10.105.82.69 <pending> 8080:32335/TCP 9s
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService>
```

Figure 13 : Réponse du service PHP avec paramètre

L'URL `http://127.0.0.1:61891/customer/Jean` appelle le service PHP qui retourne 'Customer: Jean, PHP says: JOAO GABRIEL MARQUES DINIS'. Cela démontre la communication inter-services dans le cluster.

14. Configuration de l'Ingress

L'addon Ingress a été activé pour permettre le routage HTTP.

```

PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> ^C
PS C:\Users\deari\Desktop\Projects\kubernetes-minikube\MyService> kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
mon-app-56cc447554-v7krg            1/1     Running   0           97s

```

Figure 14 : Activation de l'addon Ingress

La commande minikube addons enable ingress installe le contrôleur NGINX Ingress. Les pods ingress-nginx sont déployés et le contrôleur est en état Running.

```

PS C:\Users\deari\Desktop\kubernetes-minikube> kubectl get services
NAME            TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes      ClusterIP     10.96.0.1    <none>        443/TCP          10m
mon-app         LoadBalancer 10.105.82.69 <pending>     8080:32335/TCP   9m35s
PS C:\Users\deari\Desktop\kubernetes-minikube> minikube service mon-app --url
http://127.0.0.1:51780
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

```

Figure 15 : Création et vérification de l'Ingress

L'Ingress mon-ingress est créé avec le host myapp.local sur le port 80, utilisant la classe nginx.

15. Conclusion

Ce TP a permis de mettre en pratique les concepts fondamentaux de Kubernetes avec Minikube :

- Construction et publication d'images Docker vers un registry
- Création d'un cluster Kubernetes local avec Minikube
- Déploiement d'applications via kubectl create deployment et fichiers YAML
- Exposition de services via LoadBalancer
- Gestion des réplicas et des pods
- Configuration de l'Ingress pour le routage HTTP
- Communication inter-services (Java vers PHP)

Le TP est disponible sur GitHub à l'adresse :
<https://github.com/charroux/kubernetes-minikube>